

Week 3: pandas DataFrames

AI Frameworks

Today's learning goals

By the end of this lecture you should be able to:

1. Contrast a **Series** with a **DataFrame**
2. Load a CSV and inspect it with `head`, `shape`, `info`, `describe`
3. Select columns and rows; filter rows with conditions

Agenda (80 minutes)

1. Why pandas? (from lists/dicts to tables)
2. Series vs DataFrame
3. Loading CSV and first look
4. Selecting columns and rows
5. Filtering
6. Looking ahead to practice

Last week vs this week

- Week 2: lists, dicts, loops — enough Python to think about data
- Today: **pandas** — tables done properly

Remember

A DataFrame is like a spreadsheet in code: rows + named columns.

What pandas gives you

- Load CSV in one line
- Fast summaries (`describe`, counts, means)
- Select / filter without writing every loop by hand
- Plays nicely with plotting and ML libraries later

Two building blocks

Series

One column of values
(+ an index)

Like a labelled list

DataFrame

A table: many columns
(each column is a Series)

Like a spreadsheet sheet

Tiny mental picture

```
1 # DataFrame "students"
2 #   name    score  year
3 # 0  Ada      90     1
4 # 1  Alan     80     2
5 # 2  Grace    95     1
6
7 # students["score"] -> Series of scores
```

Quiz 1 — Series vs DataFrame

Quiz

A pandas **DataFrame** is best described as:

1. A single number
2. A table with rows and named columns
3. Only a plotting library
4. A neural network

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. A table with rows and named columns (built from Series columns).

Import and read CSV

```
1 import pandas as pd
2
3 df = pd.read_csv("students.csv")
4 print(type(df))
```

Convention: nickname `pd`. File path must match your notebook folder (or Colab upload).

First-look toolkit

```
1 df.head()           # first 5 rows
2 df.head(10)         # first 10
3 df.shape             # (rows, columns)
4 df.columns           # column names
5 df.info()            # types + non-null counts
6 df.describe()        # numeric summary
```

What to look for

- How many rows / columns?
- Sensible column names?
- Numeric vs text columns (`info`)
- Wild min/max in `describe`? (possible typos)

Try this

Always run `head` + `shape` before modelling anything.

Quiz 2 — Inspection

Quiz

Which call shows the number of rows and columns as a pair like (150, 5)?

1. `df.head()`
2. `df.shape`
3. `df.describe()`
4. `print(df)`

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 2

Answer

2. `df.shape` — `(n_rows, n_columns)`.

Columns

```
1 df["score"]           # one column -> Series
2 df[["name", "score"]] # several columns -> DataFrame
3
4 # optional style:
5 df.score              # works if name is a simple identifier
```

Rows by position (gentle)

```
1 df.iloc[0]           # first row
2 df.iloc[0:3]         # first three rows
3 df.iloc[0, 1]        # row 0, column 1
```

`loc` selects by labels; `iloc` by integer position. For now, filtering (next) matters more than fancy indexing.

Boolean masks

```
1 high = df[df["score"] >= 90]
2 year1 = df[df["year"] == 1]
3
4 both = df[(df["score"] >= 90) & (df["year"] == 1)]
```

Use `\&` / `|` with parentheses — not Python's `and` / `or` between Series.

Reading a filter in words

- `df[df["score"] >= 90]` — “rows where score is at least 90”
- Start simple; combine conditions only when needed

Common mistake

Writing `df["score"] >= 90 and df["year"] == 1` without `\&` and parentheses.

Quiz 3 — Filter

```
1 # df has columns: name, score
2 result = df[df["score"] > 80]
```

Quiz

What is result?

1. Only the score column
2. All rows where score is greater than 80 (full columns)
3. A single number: the mean score
4. An error always

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. A DataFrame of matching rows (all columns kept unless you select otherwise).

Key takeaways

1. Series = one column; DataFrame = table
2. `read_csv` + `head`/`shape`/`info`/`describe`
3. Select columns with `df["col"]` or a list of names
4. Filter with boolean masks

Next up

Practice

Explore a friendly CSV; answer five concrete questions with code

Week 4

Cleaning: missing values, duplicates, groupby, simple merges

DataFrame idea, inspection, or filtering—what should we redo live?